Lesson 05.06

Calculator Application

Assigning Click Event Listeners on a Loop

Math Operators

converting "number-like strings" to numbers

This Calculator application is a simplified version of a calculator.

Here's what it can do:

- Add, subtract, multiply or divide two numbers

Here's what it cannot do:

- Handle more than 2 inputted numbers
- Any other mathematical operations
- Store the results of calculations (everything is "forgotten" when you click Clear)

Here's how the user interface works:

- The user clicks a digit (0-9) or the decimal point.
- This click event calls the **onNumberClick()** function: -the function concatenates the digit (or decimal point) onto a string called **numStr**.
- If the user clicks another digit, the ****onNumberClick** function runs again, and the new digit gets concatenated onto the previous digit(s).
- With each digit-click, the ouput box is updated.
- Next, the user clicks a math operator (+,-,*,/).
- This click event calls **onOperatorClick()**, which:
    - converts **numStr** to **num**, an actual number.
    - pushes **num** into the array, **nums**.
    - saves the operator to a variable, **oper**.
    - outputs the operator symbol.
    - updates the ouput box.
- The user inputs the second number.
- The user clicks the equal button **=**.
- This click event calls **calculateAnswer()**, which:
    - pushes the second number into the **nums** array
    - runs a series of if-statements:
        - If the operator is +, add the numbers together.
        - If the operator is -, the second number is subtracted from the first.
        - If the operator is *, the numbers are multiplied.
        - If the operator is /, the first number is divided by the second.
- The **answer** is outputted
- The function is done
- The **clear** button calls **clearBox()** function, which empties the output box and resets the gloval variables.

Open the file **05.06-Calculator.html**, and take it for a spin in the browser:

- enter a number (as many digits long as you like)
- click one of the four operators (+,-,*,/)
- enter another number
- click the equals sign
- click Clear to reset

Have a look at the tags in the html file:

```
    <section>
        <div id="0" class="num-btns">0</div>
        --etc--
        <div id="4" class="num-btns">4</div>
    </section>

    <section>
        <div id="5" class="num-btns">5</div>
        --etc--
        <div id="9" class="num-btns">9</div>
    </section>

    <section>
        <div id="+" class="oper-btns">+</div>
        <div id="-" class="oper-btns">-</div>
        <div id="*" class="oper-btns">*</div>
        <div id="/" class="oper-btns">/</div>
        <div id="=" class="equals-btn">=</div>
        <div id="C" class="clear-btn">C</div>
    </section>

    <div id="num-box">
        <!-- output appears here -->
    </div>
```

The **num-btns** class appears 10 times--once for each digit. The **oper-btns** class occurs four times (+,-,*,/.

In the script tags, comment out the **FINAL.js** file and reactivate **START.js**.

Our task is to recreate **FINAL.js** from scratch, so open **05.06-Calculator-START.js**

We start by brining in the "buttons" (divs, actually). We will use **querySelectorAll** to bring in multiple tags at once as an array of objects.

Get the divs that have a class of **num-btns**. These are the digits from 0-9:

```
    const numBtnsArr = document.querySelectorAll('.num-btns');
```

Get the operator divs (+,-,*,/), which have a class of **num-btns**:

```
const operBtnsArr = document.querySelectorAll('.oper-btns');
```

Then we use a loop to go through the arrays and assign event listeners to the objects.

Loop through the arrays, adding listeners to each item:

```
for(let i = 0; i < numBtnsArr.length; i++) {
    numBtnsArr[i].addEventListener('click', onNumberClick);
}

for(let i = 0; i < operBtnsArr.length; i++) {
    operBtnsArr[i].addEventListener('click', onOperatorClick);
}
```

Get the other two "buttons" which call functions; these are the equals sign and the Clear button:

```
const equalsBtn = document.querySelector('.equals-btn');
equalsBtn.addEventListener('click', calculateAnswer);

const clearBtn = document.querySelector('.clear-btn');
clearBtn.addEventListener('click', clearBox);
```

Get the **num-box**, which is the div for the output:

```
const numBox = document.getElementById('num-box');
numBox.textContent = "";
```

Next, declare a few variables for use by the various functions. We need the following five:

- **numStr** for storing the inputted digit(s) as a string
- **num** for the numeric version of the "number-like string"
- **nums**, an array for storing the two inputted numbers
- **oper**, a string for storing the operator symbol
- **answer**, for storing the calculated result

```
let numStr = "";
let num = 0;
let nums = [];
let oper = '';
let answer = 0;
```

Next, we will write the **onNumberClick** function, which runs on click of any digit from 0-9. The function does the following:

- concatenates **this.id** onto **numStr** (**this.id** is the id of the clicked digit). We use oncatenation, so that the user isn't limited to single-digit input. If the user clicks 1, 2, 3, that need to be stored as "123".
- displays the digit in the output box

```
function onNumberClick() {
    numStr += this.id; // concat the clicked digit
    numBox.textContent += this.id; // output the clicked number
}
```

Clicking an operator calls the **onOperatorClick** function, which:

- saves the operator symbol to the **oper** variable
- converts **numStr** to a number, **num**
- pushed **num** into the **nums** array
- resets **numStr** to make way for the next number
- outputs the operator symbol

```
function onOperatorClick() {
    oper = this.id;
    num = Number(numStr);
    nums.push(num);
    numStr = '';
    numBox.textContent += ' ' + this.id + ' ';
}
```

The user inputs another number, the second of two. The **onNumberClick** function is called and the second number is concatenated.

When the user is done inputting the second number, they click the equal sign. This calls the **calculateAnswer** function, which:

- converts the second "number-like string" into a number
- pushes the second number into the **nums** array
- runs a series of if-statements to check **oper** so that it can do the correct mathematical calculation
- runs an if-statement to see if the answer is an integer. To check this, the answer is passed to the **Number.isInteger(n)** method, which returns true if **n** is an integer. If it is *not* an integer, the answer is rounded to four decimal places.

```
function calculateAnswer() {
    num = Number(numStr);
    nums.push(num);
    if(oper == '+') answer = nums[0] + nums[1];
    if(oper == '-') answer = nums[0] - nums[1];
```

```
        if(oper == '*') answer = nums[0] * nums[1];
        if(oper == '/') answer = nums[0] / nums[1];
        if(Number.isInteger(answer)) {
            numBox.textContent += ' = ' + answer;
        } else { // not an integer
            numBox.textContent += ' = ' + answer.toFixed(4);
        }
    }
```

Clicking the Clear "button" empties the output box and resets the global variables. The calculator is ready to do another simple math problem.

```
    function clearBox() {
        numBox.textContent = '';
        nums = [];
        num = 0;
        oper = '';
        total = 0;
        numStr = '';
    }
```

NEXT: Lesson 06.01